Aula 02 - Estrutura dos Programas

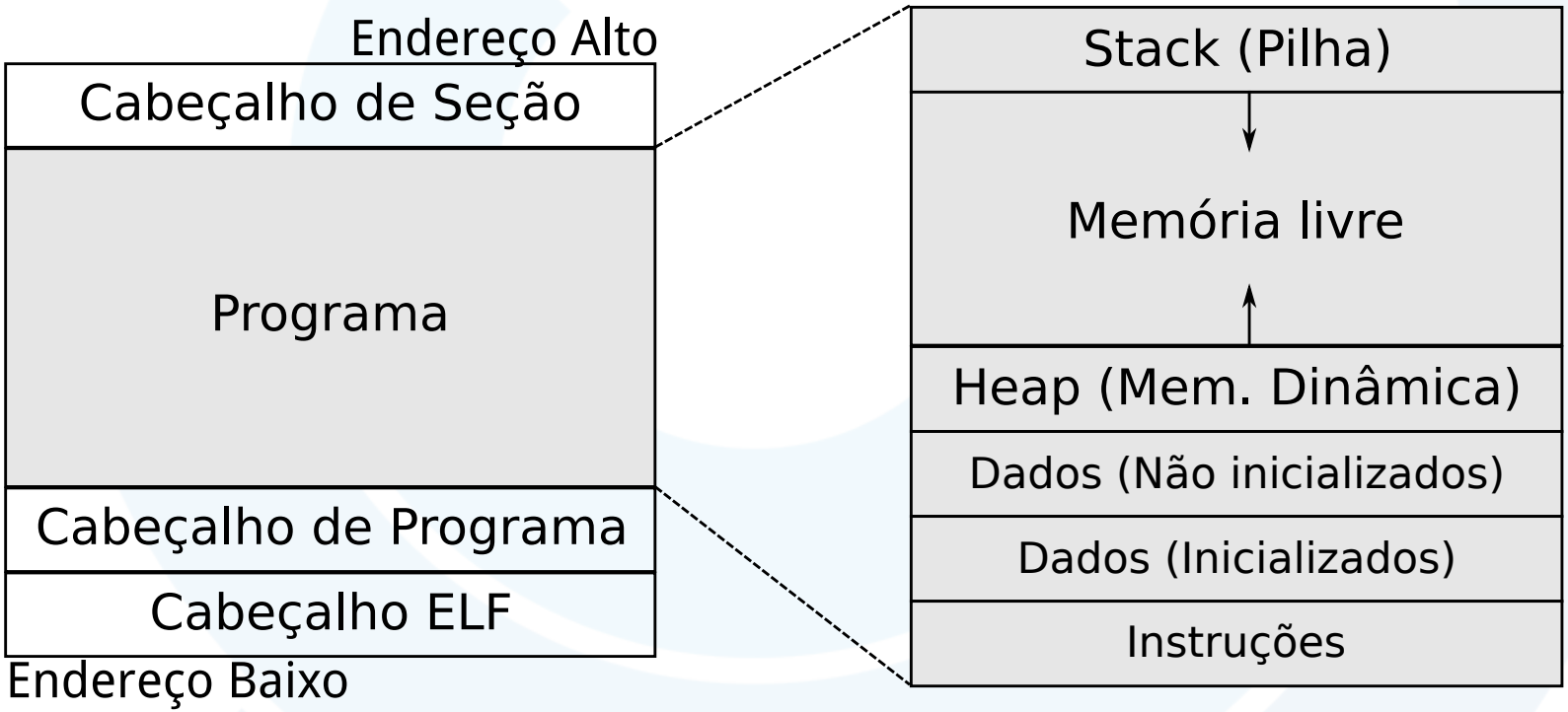
Linguagens de Montagem

Gabriela Stein

gabriela.stein1@unioeste.br

Estrutura básica de um programa

- Formato utilizado é o ELF



Formato ELF - Geral

- Cabeçalho ELF
  - Assinatura 0x7F E L F
  - Arquitetura (x86 ou x64)
  - Ponto de Entrada
  - Tamanho dos cabeçalhos
  - entre outros
- Cabeçalho de Programa
  - Informações para o Sistema Operacional
    - Como é a "imagem do programa" em memória
- Programa
- Cabeçalho de Seção
  - Tamanho, deslocamento e nomes das seções do programa

- Curiosidade: `readelf -e a02e01.x`

Formato ELF - Programa

- Instruções
  - Área que contém as instruções do programa

```
section .text
```

  - A execução é realizada a partir deste ponto (*top-down*)
- Dados Inicializados
  - Variáveis com valores pré-definidos

```
section .data
   strOla :  db "Ola", 10
```

  - Normalmente utilizada para definição de constantes
  - Pode ser alterado

Formato ELF - Programa

- Dados Não Inicializados
  - Variáveis
  - É realizado uma reserva de bytes em memória
    - Discutido na próxima aula
- Memória Dinâmica
  - Variáveis Dinâmicas
    - Calloc/Malloc
  - Será discutido no futuro
- Área Livre de Memória
  - Área entre `Memória Dinâmica` e `Pilha`
- Pilha
  - Passagem de Parâmetros, armazenamento temporário
  - Será discutido no futuro

Dados

- Comportamento
  - Não existe “tipos de dados” em memória
  - É a instrução que determina o comportamento
- Normalmente
  - É feito a definição do valor em memória (inicializadas)

    ou
  - É feito a reserva de espaço em memória (não inicializadas)

Dados Inicializados (.data)

- Variáveis com valores pré-definidos

```
section .data
  simbolo: tamanho valor
```

  - Uma linha é uma variável inicializada
  - Uma variável é definida por um símbolo (*label*)

```
char c = 10;
```

    - c é o nome da variável, ou seja, o símbolo para referência
    - 10 é o seu valor pré-definido
  - Uma variável tem um tamanho em memória especificado por uma pseudo-instrução

```
db: byte
dw: word (2B)
dd: dualword (4B)
dq: quadword (8B)
```

Dados Inicializados (.data) - Exemplos

```
; Aula 02 - Estrutura dos Programas
; arquivo: a02e01.asm
; objetivo: dados inicializados
; nasm -f elf64 a02e01.asm ; ld a02e01.o -o a02e01.x

section .data
    v1: db    0x55               ; byte 0x55
    v2: db    0x55,0x56,0x57     ; 3 bytes em sucessao
    v3: db    'a',0x55           ; caracteres sao aceitos com aspas
    v4: db    'hello',13,10,'$'  ; strings tambem
    v5: dw    0x1234             ; 0x34 0x12
    v6: dw    'a'                ; 0x61 0x00
    v7: dw    'ab'               ; 0x61 0x62
    v8: dw    'abc'              ; 0x61 0x62 0x63 0x00 (string)
    v9: dd    0x12345678         ; 0x78 0x56 0x34 0x12
```

code: a02e01.asm (parcial)

Debugger?

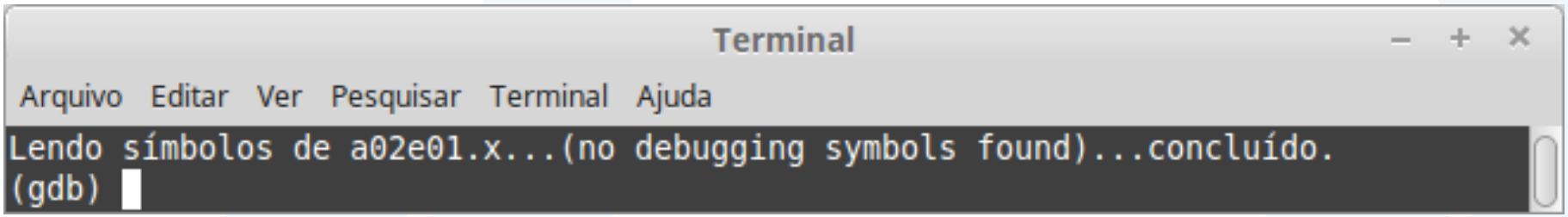
- Almirante Grace Hoper em 1947
  - Marinha dos EUA
  - Popularizou o termo na computação quando seus colegas encontraram uma mariposa impedindo o correto funcionamento do `MARK II`
  - Os termos *bug* e *debug* são antigos (registros em 1878 - Thomas Edison)
  - †1992
- gdb - GNU Debugger
  - Nativo na maioria das distros Linux
  - Permite depurar outro programas para encontrar *bugs* ou validá-los

▸ GDB: The GNU Project Debugger

GDB

- Com o arquivo executável que se deseja verificar, faça:

```
gdb nome.x
```



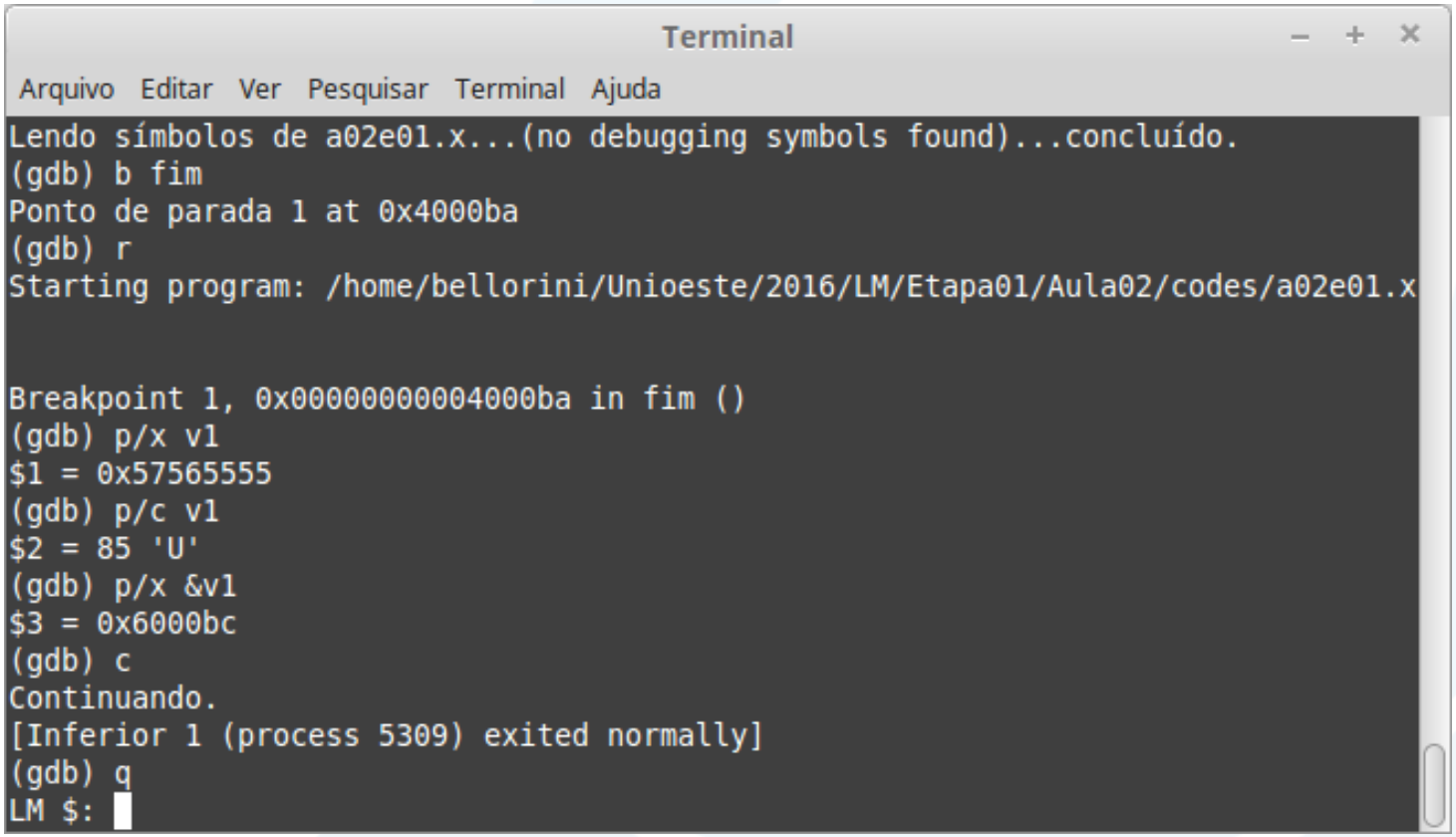
- Comandos:
  - Sair: quit (q)
  - BreakPoint: break (b) nomeLabel
  - Executar: run (r) args
  - Continuar: continue (c) - quando a execução encontra um BreakPoint
  - Mostrar valor de variável: print/x (p/x) nomeVar
    - Será mostrado o valor de 4 bytes em hexadecimal
  - Mostrar valor de variável: print/c (p/c) nomeVar
    - Será mostrado o valor de 1 bytes em hexadecimal
  - Mostrar endereço de variável: print/x (p/x) &nomeVar

GDB - Passos para exemplo: a02e01.x

- Chamar GDB para o exemplo
  $: gdb a02e01.x
- Configurar breakPoint
  (gdb) b fim
- Executar programa exemplo
  (gdb) r
- Mostrar o valor (4 bytes) da variável V1
  (gdb) p/x (casting de tipo) v1
- Mostrar o valor (1 byte) da variável V1
  (gdb) p/c (casting de tipo) v1
- Mostrar endereço da variável V1
  (gdb) p/x &v1
  Importante: o casting de tipo (*type-casting*) informa ao depurador qual o tamanho da variável a ser impressao

GDB - Passos para exemplo: a02e01.x

- Continuar a execução
  (gdb) c



GDB - mais dicas 1:

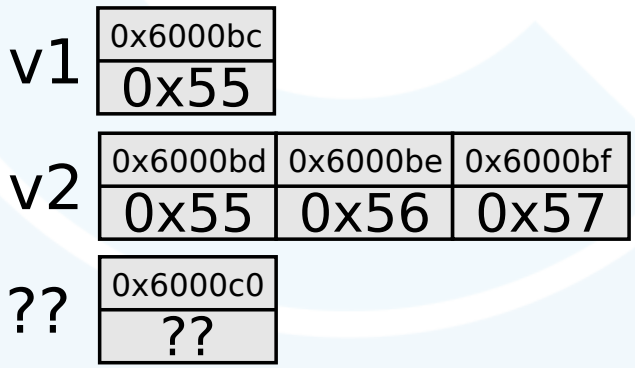
- Chamar GDB para o exemplo
  $: gdb a02e01.x
- Acessar memória
  (gdb) x /[bhwg] end
  inteiro sinalizado (default)
  1 byte: (gdb) x /b end
  2 bytes: (gdb) x /h end
  4 bytes: (gdb) x /w end
  8 bytes: (gdb) x /g end

GDB - mais dicas 2:

- Chamar GDB para o exemplo
  $: gdb a02e01.x
- Acessar memória
  (gdb) x /[bhwg] end
  inteiro sinalizado (default)
- interpretar
  x /[bhwg][cduxto] end
  c: caractere (apenas 1 byte)
  d: inteiro sinalizado
  u: inteiro não sinalizado
  t: binário
  x: hexadecimal
  o: octal

Atividade para Praticar

- AT0201: Debuggar as outras variáveis do código exemplo a02e01.asm
  - Crie um diagrama da memória com seu conteúdo
    - Posicionar cada variável em seu devido endereço a nível de byte
    - Ao todo são 28 bytes para o exemplo
  - Exemplo para v1 e v2:



Fim do Documento

Dúvidas?

Próxima aula:

- Aula 03: Registradores e Instrução MOV
  - Registradores
    - Topo da hierarquia de memórias
    - Visíveis e Não-Visíveis
    - Modos de acesso
  - Transferência de dados
    - Memória para Registrador
    - Registrador para Memória
    - Registrador para Registrador